

Aeris Checkpoint Report

Infrastructure freeze before the real geometry build. Prepared as a reading-first briefing for the next development phase.

1. Executive summary

The project has successfully crossed the setup phase. The repository now contains a working Python package, a clean Typer-based CLI, config-driven execution, reproducible run folders, logging, manifest tracking, a smoke pipeline, and the first geometry scaffold. This is the correct moment to freeze infrastructure, push to GitHub, and begin the real geometry build.

- Package name: aeris
- Repository path: ~/Desktop/Start_Up/Code/v.0.1_Project
- GitHub remote: git@github.com:karmix96/aeris.git
- Python requirement: >=3.13
- CLI entrypoint: aeris = "aeris.cli:main"
- Current strategic state: infrastructure is sufficient; geometry mathematics is the next real priority

2. Development principles

- Python-first and OS-agnostic
- Config-driven execution instead of hardcoded parameters
- One clean CLI rather than scattered scripts
- Reproducible runs with run folders, manifests, and logs
- Step-by-step educational workflow
- Avoid premature complexity and fake end-to-end demos

3. Architecture at the checkpoint

The codebase is now organized into four layers. This separation is important because it prevents the CLI, workflow orchestration, and engineering logic from collapsing into one swollen file.

Layer	Primary location	Responsibility
CLI	src/aeris/cli.py	Front door of the software. Registers command groups and routes execution.
Commands	src/aeris/commands/	Defines user-facing commands/options and hands off to pipeline/domain logic.
Pipeline	src/aeris/pipeline/	Orchestrates workflows: load config, create run folder, log, write manifest, call domain m
Domain + common	src/aeris/geometry/ and src/aeris/common/	Geometry logic plus shared utilities such as paths, config loading, and logger setup.

4. What exists in the repository

Tracked checkpoint files (the meaningful code/reporting/config set) are listed below. Runtime outputs such as data/runs are intentionally not committed.

Root

.gitignore
README.md
pyproject.toml

Documents

documents/Calendar.docx
documents/Guidelines.docx
documents/Master_Plan.docx
documents/SW_Plan.docx

Configs

configs/smoke/dev.yaml
configs/geometry/wing_basic.yaml

Package root

src/aeris/__init__.py
src/aeris/cli.py

Commands

src/aeris/commands/__init__.py
src/aeris/commands/version.py
src/aeris/commands/geometry.py
src/aeris/commands/pipeline.py

Common

src/aeris/common/__init__.py
src/aeris/common/paths.py
src/aeris/common/config.py
src/aeris/common/logging_utils.py

Pipeline

src/aeris/pipeline/__init__.py
src/aeris/pipeline/smoke.py
src/aeris/pipeline/geometry_run.py

Geometry

src/aeris/geometry/__init__.py
src/aeris/geometry/params.py
src/aeris/geometry/validation.py
src/aeris/geometry/export.py

Local-only / ignored items

- .venv/
- data/runs/
- __pycache__/
- *.egg-info/

5. Execution logic that already works

The important point is not just that files exist, but that the system already executes end-to-end in a controlled way.

- aeris version works
- aeris pipeline smoke --config configs/smoke/dev.yaml works
- aeris geometry generate --config configs/geometry/wing_basic.yaml works
- Each run creates a timestamped folder under data/runs/
- Each run writes manifest.json, logs/app.log, input_config.yaml, and artifacts/
- Success and failure are recorded explicitly in manifests
- Bad YAML triggers a failed manifest and a nonzero exit code

Current run folder model

```
data/runs/<timestamp>_<name>/
■■■ input_config.yaml
■■■ manifest.json
■■■ logs/
■   ■■■ app.log
■■■ artifacts/
    ■■■ geometry/
```

6. Current geometry status

Geometry is no longer absent, but it is still an honest scaffold. The code already parses a simple wing config, validates the numbers, computes basic planform quantities, and exports geometry artifacts. However, the exported station coordinates are still placeholders in the important places.

- Already implemented: span, aspect ratio, taper ratio, sweep angle, dihedral angle, root chord, tip chord
- Already exported: geometry_summary.json and planform_sections.csv
- Still placeholder: sweep does not yet affect x_le_m
- Still placeholder: dihedral does not yet affect z_m
- Still limited: only root and tip stations; no rich spanwise station generation yet

What this means in practice

The project is now ready for real geometry mathematics. It is not ready for ML, optimization, or elaborate CAD theatrics. The next useful progress is to improve the planform core until the generated coordinates are physically meaningful and exportable to downstream aero tools.

7. Why the current checkpoint matters

This checkpoint is worth preserving because it draws a clean line between infrastructure and engineering. The project now has enough architecture to support real development without drowning in setup work, but it has not yet overcomplicated itself.

- The CLI is stable enough
- Run tracking is stable enough
- Failure handling is stable enough
- Geometry is present but still simple enough to reshape cleanly
- GitHub push is complete, so the checkpoint is frozen

8. Recommended next phase

The next chat should not restart infrastructure. It should begin the geometry build deliberately. The recommended sequence is below.

Step	Focus	Outcome
9	Real planform geometry core	Sweep- and dihedral-aware section coordinates, richer station table, area and MAC check
10	Airfoil section input and placement	Imported airfoil coordinates, spanwise sections scaled and placed in 3D.

11	Physics-facing export	Minimal but valid export for AVL or another first analysis target.
12	First external wrapper	Run one real aero tool, capture outputs, store results in the run folder.

9. Practical reading notes for Mike

- Do not memorize every file. Understand the role of each layer: CLI -> commands -> pipeline -> domain/common.
- The key idea is reproducibility: every run creates a traceable folder with logs, config, and manifest.
- The geometry code exists to generate inspectable engineering outputs, not just pretty abstractions.
- From here on, every step should make geometry more physically meaningful, not the architecture more fancy.

One-line takeaway

Aeris has a solid software skeleton; the next chat should build the first real geometric muscle.